Actividad 7. Máquinas de Soporte Vectorial (SVM)

Se trabajó con la base de datos recursos_humanos.csv de la misma manera que se hace en los otros ejercicios, que contiene información de 14,999 empleados. La variable que se busca predecir es left: el valor 0 indica que el empleado permaneció y el valor 1 indica que abandonó la empresa, base de datos ya usada en una actividad anterior.

```
import pandas as pd
import pylab as pl
import numpy as np
import scipy.optimize as opt
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

```
import os
os.chdir("D:/gus6e/Documents")
cell_df = pd.read_csv("recursos_humanos.csv")
cell_df
```

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	s
0	0.38	0.53	2	157	3	0	1	0	s
1	0.80	0.86	5	262	6	0	1	0	s
2	0.11	0.88	7	272	4	0	1	0	s
3	0.72	0.87	5	223	5	0	1	0	s

Se revisó cuántos empleados pertenecían a cada grupo. De los 14,999 registros, 11,428 corresponden a trabajadores que permanecieron y 3,571 a quienes abandonaron la empresa. Esto representa aproximadamente el 76.2 % y el 23.8 %, respectivamente. También se realizó una gráfica de dispersión para comparar el nivel de satisfacción y la última evaluación. Al igual que en el ejemplo de la lección, se tomaron las primeras 50 observaciones de cada categoría para que la gráfica fuera más fácil de observar.

```
# No necesitamos transformar la variable left
cell_df.dtypes
```

```
satisfaction_level       float64
last_evaluation          float64
number_project             int64
average_montly_hours       int64
time_spend_company         int64
Work_accident              int64
left                       int64
promotion_last_5years      int64
sales                     object
salary                    object
dtype: object
```

```
cell_df["left"].value_counts()
```

```
left
0    11428
1     3571
Name: count, dtype: int64
```

Antes de aplicar SVM, se transformaron las variables cualitativas de departamento y categoría salarial en columnas numéricas con pd.get_dummies, como solicita la actividad. Así, cada categoría quedó indicada con valores de cero y uno, sin perder la información original de los empleados. Después de la transformación se obtuvieron 20 características para hacer el pronóstico y se dejó left como la variable dependiente.

```
cell_df = pd.get_dummies(cell_df, columns = ["sales", "salary"], dtype = int)
cell_df
```

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	sales_IT	sales_Randc
0	0.38	0.53	2	157	3	0	1	0	0	
1	0.80	0.86	5	262	6	0	1	0	0	
2	0.11	0.88	7	272	4	0	1	0	0	
3	0.72	0.87	5	223	5	0	1	0	0	
4	0.37	0.52	2	159	3	0	1	0	0	
...	...	...	...	...	...	...	...	...	...	...
14994	0.40	0.57	2	151	3	0	1	0	0	
14995	0.37	0.48	2	160	3	0	1	0	0	
14996	0.37	0.53	2	143	3	0	1	0	0	
14997	0.11	0.96	6	280	4	0	1	0	0	
14998	0.37	0.52	2	158	3	0	1	0	0	

14999 rows × 21 columns

```
cell_df.dtypes
```

```
satisfaction_level      float64
last_evaluation         float64
number_project            int64
average_montly_hours      int64
time_spend_company        int64
Work_accident             int64
left                      int64
```

Se separaron las observaciones en dos grupos mediante el mismo procedimiento de la lección. Se utilizaron 11,999 empleados para entrenar los modelos y 3,000 para evaluar sus predicciones. En este último grupo había 2,287 empleados que permanecieron y 713 que abandonaron la empresa. Se mantuvo la misma división para probar los cuatro kernels y poder compararlos de forma consistente.

```
feature_df = cell_df[["satisfaction_level", "last_evaluation", "number_project", "average_montly_hours", "time_spend_company", "Work_accident", "prom
x = np.asarray(feature_df)
x[0:5]
```

```
array([[3.80e-01, 5.30e-01, 2.00e+00, 1.57e+02, 3.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        1.00e+00, 0.00e+00],
       [8.00e-01, 8.60e-01, 5.00e+00, 2.62e+02, 6.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 1.00e+00],
       [1.10e-01, 8.80e-01, 7.00e+00, 2.72e+02, 4.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 1.00e+00],
       [7.20e-01, 8.70e-01, 5.00e+00, 2.23e+02, 5.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        1.00e+00, 0.00e+00],
       [3.70e-01, 5.20e-01, 2.00e+00, 1.59e+02, 3.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        1.00e+00, 0.00e+00]])
```

```
cell_df["left"] = cell_df["left"].astype("int")
y = np.asarray(cell_df["left"])
y[0:5]
```

```
array([1, 1, 1, 1, 1])
```

```
# Creación de grupos de entrenamiento y prueba
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size= 0.2, random_state = 4)
print("Grupo de entrenamiento", x_train.shape, y_train.shape)
print("Grupo de prueba",x_test.shape,y_test.shape)
```

```
Grupo de entrenamiento (11999, 20) (11999,)
Grupo de prueba (3000, 20) (3000,)
```

Se aplicó el algoritmo de Máquinas de Soporte Vectorial con las cuatro opciones vistas en la lección: RBF, lineal, polinomial y sigmoide. En cada caso se construyó una matriz de confusión, se representó con un mapa de calor y se obtuvo un reporte de clasificación. La matriz permite contar los aciertos y errores; el reporte presenta precision, recall, F1-score y support.

Para la transformación RBF de Kernel se obtuvo una precisión global de 77,7 %, lo que significa que esa proporción de los 3,000 empleados de prueba fue clasificada correctamente. En la matriz de confusión se identificaron 2287 aciertos de permanencia, 44 aciertos de abandono, 0 empleados que permanecieron pero fueron pronosticados como abandonos y 669 empleados que abandonaron pero fueron pronosticados como permanencias.

Para la categoría de abandono, la precision fue de 100,0 % y el recall de 6,2 %. Esto significa que los pocos casos que señaló como abandono fueron correctos, pero dejó

sin detectar a la mayoría de los que realmente abandonaron. El F1-score de esta categoría fue de 0.12, lo que muestra esa limitación.

Para los empleados que permanecieron, la precision fue de 77,4 %, el recall de 100,0 % y el F1-score de 0.87. El support del reporte es de 2,287 para permanencia y 713 para abandono.

Kernel RBF

```
from sklearn import svm
clf = svm.SVC(kernel = "rbf")
clf.fit(x_train, y_train)

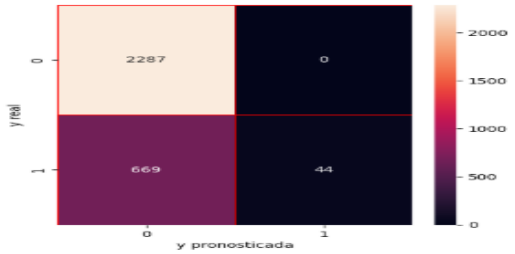
# Predicción para la base de prueba
yhat = clf.predict(x_test)
yhat[0:5]
```

```
array([0, 0, 0, 0, 0])
```

```
# Creación de matriz de confusión
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(x)
cm = confusion_matrix(y_test,yhat)
cm
```

```
array([[2287,    0],
       [ 669,   44]])
```

```
# Visualizar la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize = (5,5))
sns.heatmap(cm,annot=True, linewidths = 0.5, linecolor = "red", fmt = ".0f", ax = ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.show()
```




```
from sklearn.metrics import classification_report
cnf_matrix = confusion_matrix(y_test, yhat,labels = [0,1])
print(classification_report(y_test,yhat))
```

```
              precision    recall  f1-score   support

           0       0.77      1.00      0.87      2287
           1       1.00      0.06      0.12       713

    accuracy                           0.78      3000
   macro avg       0.89      0.53      0.49      3000
weighted avg       0.83      0.78      0.69      3000
```

Para la transformación lineal de Kernel se obtuvo una precisión global de 78,0 %, lo que significa que esa proporción de los 3,000 empleados de prueba fue clasificada correctamente. En la matriz de confusión se identificaron 2149 aciertos de permanencia, 191 aciertos de abandono, 138 empleados que permanecieron pero fueron pronosticados como abandonos y 522 empleados que abandonaron pero fueron pronosticados como permanencias.

En la categoría de abandono, la precisión fue de 58,1 %, el recall de 26,8 % y el F1-score de 0.37. Estos indicadores ayudan a valorar el modelo más allá de la precisión global, porque muestran qué tan bien identifica los casos que interesan al departamento de recursos humanos.

Para los empleados que permanecieron, la precision fue de 80,5 %, el recall de 94,0 % y el F1-score de 0.87. El support del reporte es de 2,287 para permanencia y 713 para abandono.

Kernel Lineal

```
from sklearn import svm
clf = svm.SVC(kernel = "linear")
clf.fit(x_train, y_train)

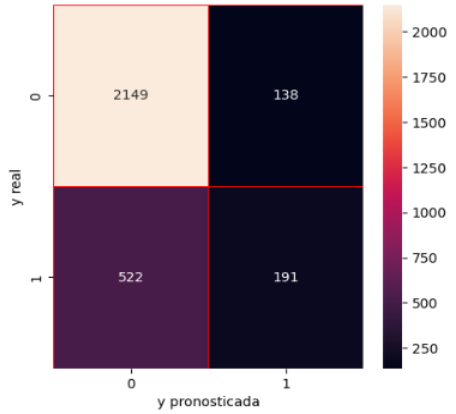
# Predicción para la base de prueba
yhat = clf.predict(x_test)
yhat[0:5]
```

```
array([0, 1, 0, 0, 0])
```

```
# Creación de matriz de confusión
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(x)
cm = confusion_matrix(y_test,yhat)
cm
```

```
array([[2149,  138],
       [ 522,  191]])
```

```
# Visualizar la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize = (5,5))
sns.heatmap(cm,annot=True, linewidths = 0.5, linecolor = "red", fmt = ".0f", ax = ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.show()
```



Para la transformación polinomial de Kernel se obtuvo una precisión global de 76,2 %, lo que significa que esa proporción de los 3,000 empleados de prueba fue clasificada correctamente. En la matriz de confusión se identificaron 2287 aciertos de permanencia, 0 aciertos de abandono, 0 empleados que permanecieron pero fueron pronosticados como abandonos y 713 empleados que abandonaron pero fueron pronosticados como permanencias.

El reporte confirma que este kernel no identificó ningún caso real de abandono. Aunque acertó la mayor parte de los casos de permanencia, el resultado no es conveniente para el objetivo de la actividad, porque precisamente se necesita reconocer a quienes podrían irse. Para los empleados que permanecieron, la precisión fue de 76,2 %, el recall de 100,0 % y el F1-score de 0.87. El support del reporte es de 2,287 para permanencia y 713 para abandono.

Kernel Polinomial

```
from sklearn import svm
clf = svm.SVC(kernel = "poly")
clf.fit(x_train, y_train)

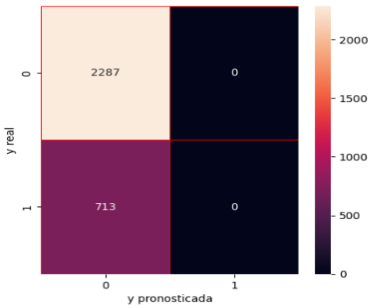
# Predicción para la base de prueba
yhat = clf.predict(x_test)
yhat[0:5]
```

```
array([0, 0, 0, 0, 0])
```

```
# Creación de matriz de confusión
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(x)
cm = confusion_matrix(y_test,yhat)
cm
```

```
array([[2287,    0],
       [ 713,    0]])
```

```
# Visualizar la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize = (5,5))
sns.heatmap(cm,annot=True, linewidths = 0.5, linecolor = "red", fmt = ".0f", ax = ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.show()
```



Para la transformación sigmoide se obtuvo una precisión global de 69,7 %, lo que significa que esa proporción de los 3,000 empleados de prueba fue clasificada correctamente. En la matriz de confusión se identificaron 1885 aciertos de permanencia, 206 aciertos de abandono, 402 empleados que permanecieron pero fueron pronosticados como abandonos y 507 empleados que abandonaron pero fueron pronosticados como permanencias.

En la categoría de abandono se obtuvo una precision de 33,9 % y un recall de 28,9 %. El resultado señala que identificó algunos casos adicionales de abandono, pero también confundió a varias personas que permanecieron. Su F1-score para esta categoría fue de 0.31. Para los empleados que permanecieron, la precision fue de 78,8 %, el recall de 82,4 % y el F1-score de 0.81. El support del reporte es de 2,287 para permanencia y 713 para abandono.

Kernel Sigmoide

```
from sklearn import svm
clf = svm.SVC(kernel = "sigmoid")
clf.fit(x_train, y_train)

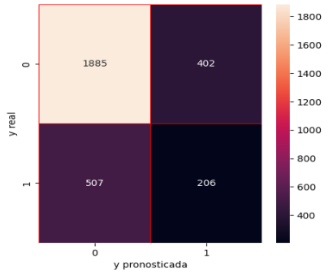
# Predicción para la base de prueba
yhat = clf.predict(x_test)
yhat[0:5]
```

```
array([0, 0, 0, 0, 0])
```

```
# Creación de matriz de confusión
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(x)
cm = confusion_matrix(y_test,yhat)
cm
```

```
array([[1885,  402],
       [ 507,  206]])
```

```
# Visualizar la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize = (5,5))
sns.heatmap(cm,annot=True, linewidths = 0.5, linecolor = "red", fmt = ".0f", ax = ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.show()
```



Después de comparar los cuatro métodos, se recomienda el kernel lineal. Esta elección se basa en los resultados obtenidos con la misma muestra de prueba y, especialmente, en la capacidad para identificar a los empleados que abandonaron la empresa. Aunque otros kernels pueden alcanzar una precisión global similar, es importante revisar también cuántos abandonos dejan sin detectar y cuántas falsas alarmas producen.

El kernel lineal alcanzó una precisión global del 78.0 % y obtuvo el F1-score más alto para el abandono, de aproximadamente 0.37. Detectó 191 de los 713 casos reales de salida, mientras que 522 pasaron inadvertidos. Aunque el sigmoide detectó 206 abandonos, registró muchas más falsas alarmas, 402 frente a 138 del lineal. Así, el lineal ofrece un equilibrio relativamente mejor entre los aciertos de abandono y las falsas alarmas dentro de los cuatro modelos probados. Sin embargo, su recall de abandono fue de solo 26.8 %, por lo que debe reconocerse que aún falla al identificar a muchas personas que dejan la empresa.

Pronóstico para el empleado

Por último, se consideró al empleado con nivel de satisfacción de 0.5, última evaluación de 0.75, cuatro proyectos, 200 horas trabajadas al mes, cuatro años en la empresa, ningún accidente laboral, ninguna promoción en los últimos cinco años, departamento de ventas y salario medio. Estos datos se colocaron en el mismo orden que las variables utilizadas para entrenar el modelo, incluyendo las columnas numéricas creadas para el departamento y el salario. Con el kernel lineal, el resultado pronosticado fue 0. Esto indica que, de acuerdo con el modelo, el empleado permanecería en la empresa.

Decisión de modelo y pronóstico

```
feature_df.columns
```

```
Index(['satisfaction_level', 'last_evaluation', 'number_project',
       'average_montly_hours', 'time_spend_company', 'Work_accident',
       'promotion_last_5years', 'sales_IT', 'sales_RandD', 'sales_accounting',
       'sales_hr', 'sales_management', 'sales_marketing', 'sales_product_mng',
       'sales_sales', 'sales_support', 'sales_technical', 'salary_high',
       'salary_low', 'salary_medium'],
      dtype='object')
```

```
nuevo_empleado = np.asarray([[0.5, 0.75, 4, 200, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]])
pronostico = clf.predict(nuevo_empleado)
print("Pronóstico para el empleado (1 = abandonó, 0 = permaneció):", pronostico[0])
```

```
Pronóstico para el empleado (1 = abandonó, 0 = permaneció): 0
```